

MACHINE LEARNING



Linear Regression

Predicting continuous outcomes by learning the relationship between inputs and a target

Presented By : Nancy Ahmed



Table of content

01. Introduction to Regression

Machine Learning Types
Regression vs. Classification
Features and Target

02. Understanding the Best-Fit Line

Simple Regression | Multiple Linear
Regression | Best-Fit Line Concept |
Prediction Examples

03. Measuring Model Performance

Absolute Error
Squared Error
Mean Squared Error (Cost Function)

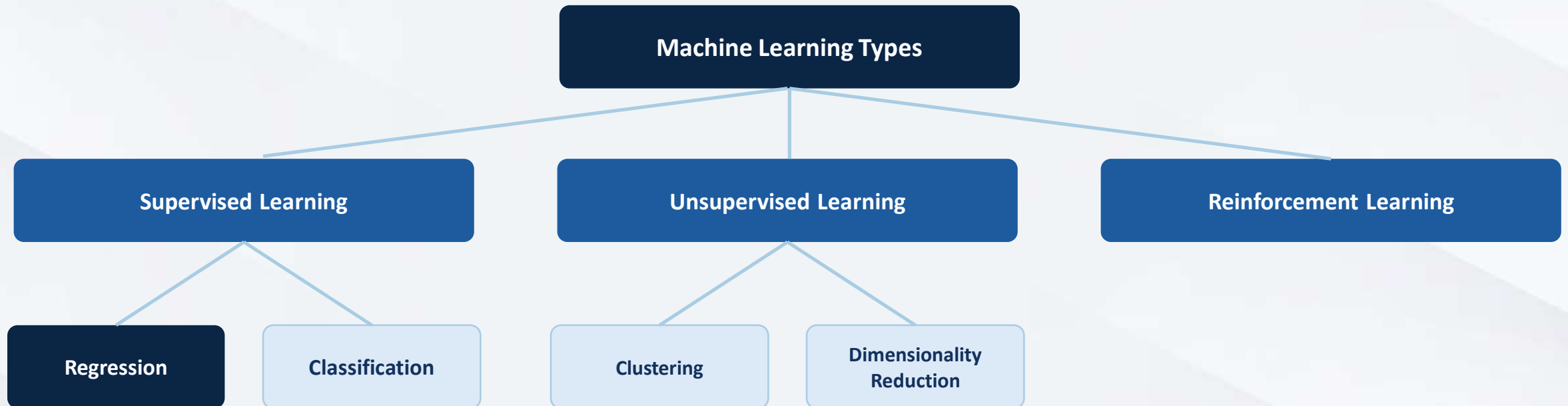
04. Training with Gradient Descent

Cost Minimization | Derivatives and
Gradient | Learning Rate | Convergence |
Global vs. Local Minima

05. Beyond Linear Relationships

linear regression assumptions,
regularization (L1 & L2) and
Polynomial Regression

Machine Learning Types



What Is Regression?

The goal of regression is to build a predictive model that produces **continuous** predictions or estimates — as opposed to classification, which deals with discrete categories or classes.



EXAMPLES

- 1 House prices
- 2 Video watch time
- 3 Estimated medical expenses

Independent vs. Dependent Variables

Regression predicts numerical values by modeling the relationship between a **dependent variable** (target) and one or more **independent variables** (features).

TARGET

(Dependent Variable)

What you want to predict.

FEATURE

(Independent Variables)

The inputs used to make the prediction.

Example: Car Price Prediction

TARGET (DEPENDENT VARIABLE)

Car Price

(in USD)

FEATURES (INDEPENDENT VARIABLES)

- Engine size (liters)
- Year of manufacture
- Brand
- Fuel type

Predicting House Prices

Imagine you're trying to predict a house price based on the number of rooms.

Number of Rooms	House Price
1	150
2	200
3	250
4	?
5	350
6	400
7	450

OBSERVATION

As the number of rooms increases, the price of the house increases too.

From Pattern to Formula

Each extra room adds **\$50** to the price, and the base price — the price when rooms = 0 — is **\$100**.

This pattern-matching is exactly what Linear Regression does.

$$\text{new price} = 50 \times \text{number of rooms} + 100$$

$$y = w \times x + b$$

Linear Regression Equation

$$y = w \times x + b$$

y

Target

The value we want to predict.

x

Feature

The input variable used to make the prediction.

w

Weight

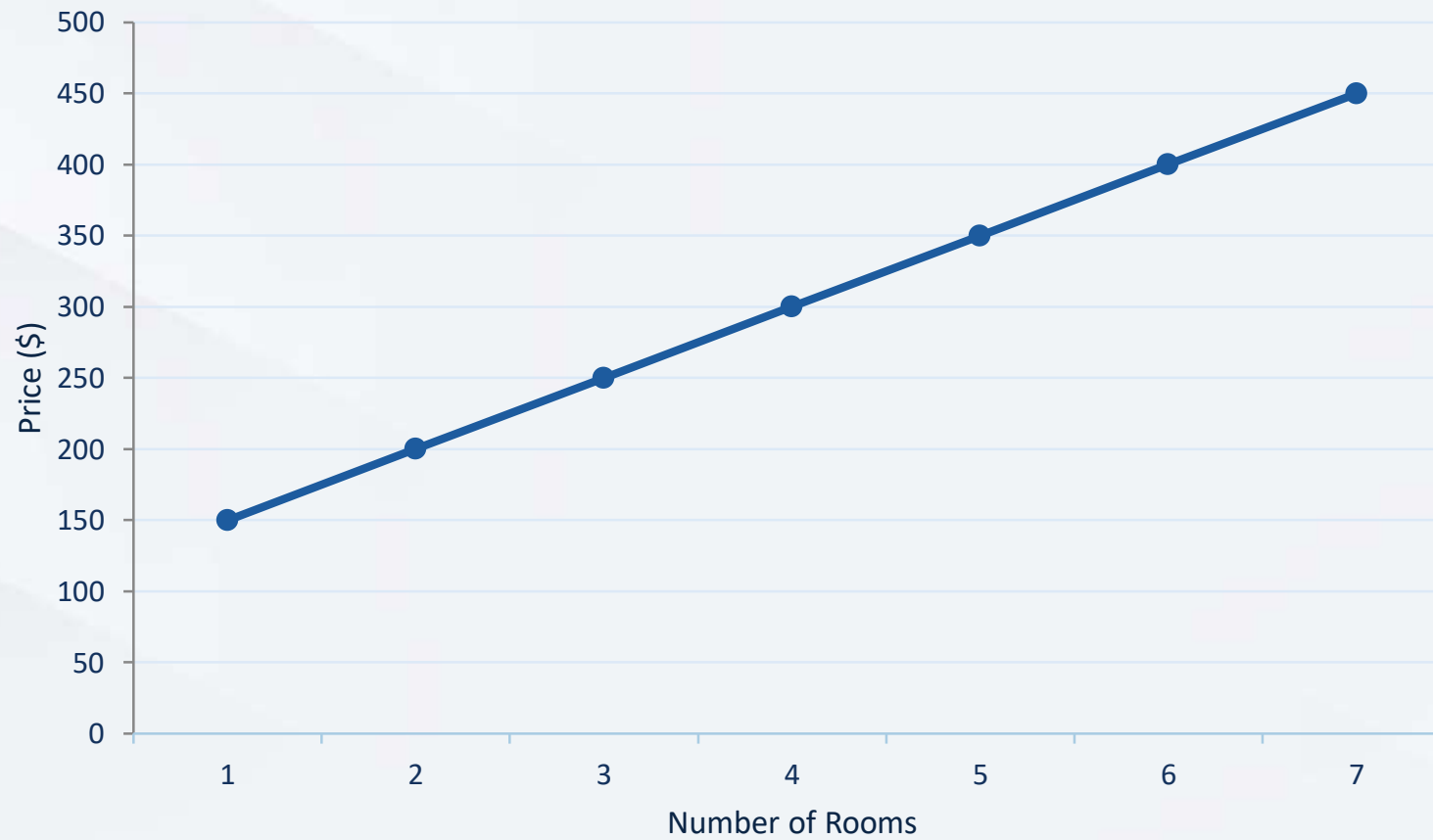
The slope — how much y changes when x changes by 1 unit.

b

Bias

The intercept — the value of y when x = 0.

The Best-Fit Line



AT 4 ROOMS

\$300

$$y = 50x + 100$$

$$y = 50(4) + 100$$

House Pricing With a Harder Pattern

This dataset is more realistic — prices don't follow a clean pattern. How can we compute w ?

Number of Rooms	House Price
1	155
2	197
3	244
4	?
5	356
6	407
7	448

KEY IDEA

With noisy data there's no single perfect slope. Linear regression finds the line that minimizes overall error across every point.

House Pricing With Multiple Features

When we add an extra feature, how can we get the weights?

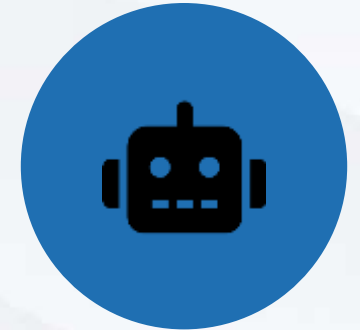
Rooms	Area	House Price
1	95	155
2	95	197
3	100	244
4	160	?
5	170	356
6	180	407
7	180	448

$$y = w_1x_1 + w_2x_2 + b$$

Each feature gets its own weight. More features means more dimensions — but the same core idea: a weighted sum plus a bias.

Why Use Linear Regression in Machine Learning?

- 1 Finds the best-fitting line through data, even when the slope isn't fixed or there are multiple features
- 2 Learns the relationship between inputs and output by minimizing error
- 3 Makes reliable predictions even with noisy or complex data

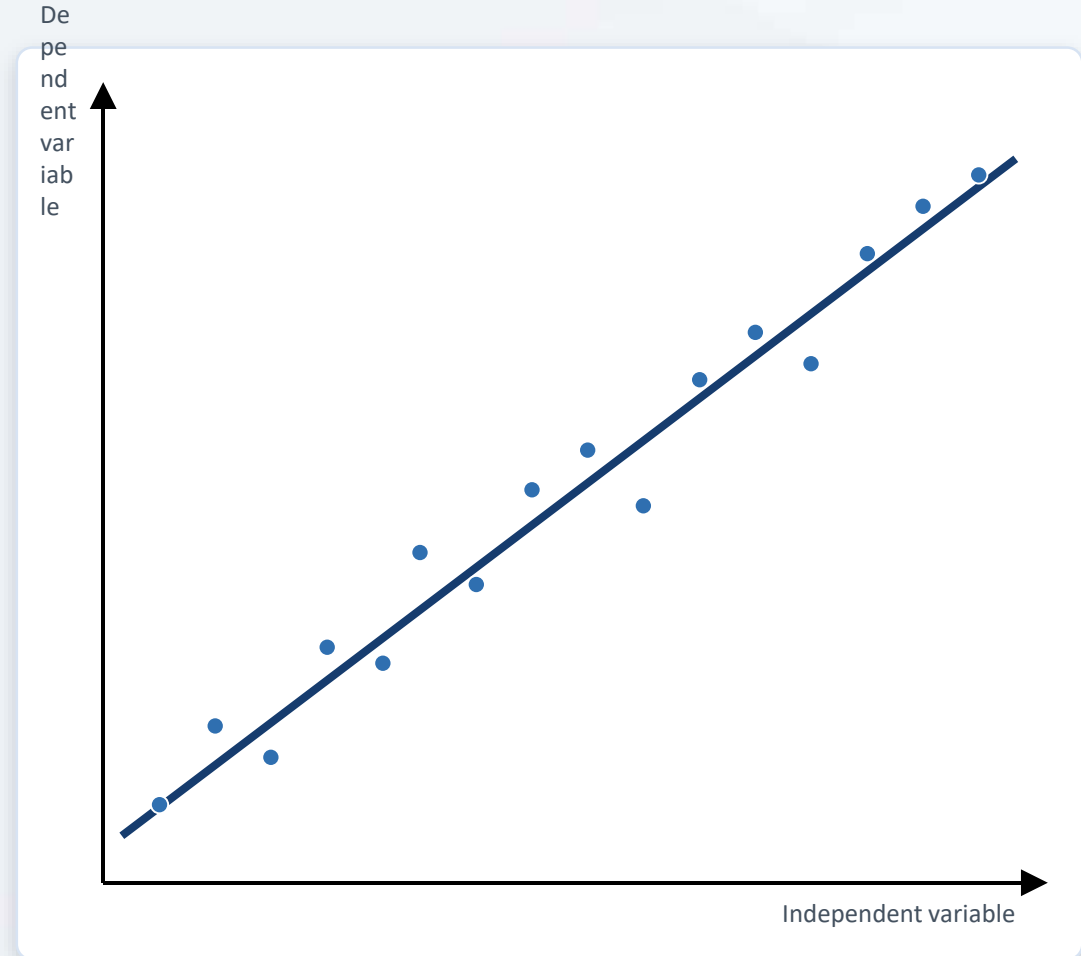


Finding the best-fitting line through data

What Is Linear Regression?

Linear regression is a method used to find the **best-fitting straight line** that represents the relationship between one or more input variables (features) and an output (target).

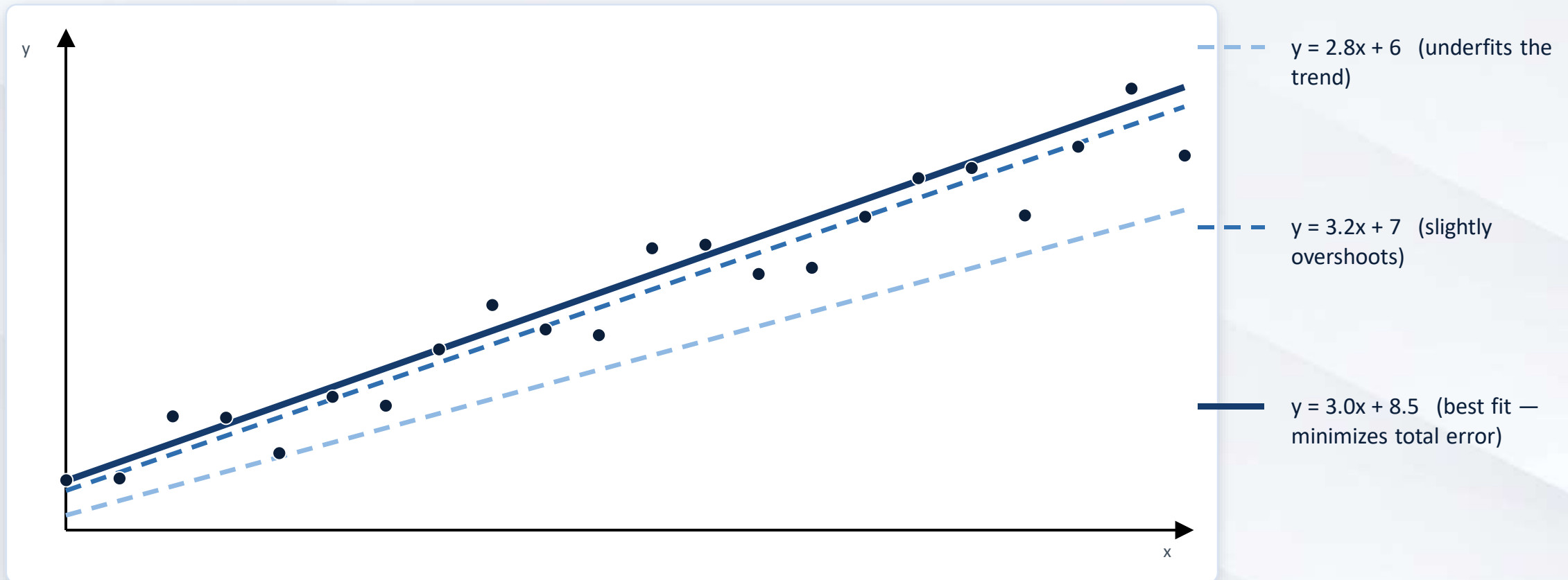
Best line = the right weight & bias



Data points cluster around a single line of best fit.

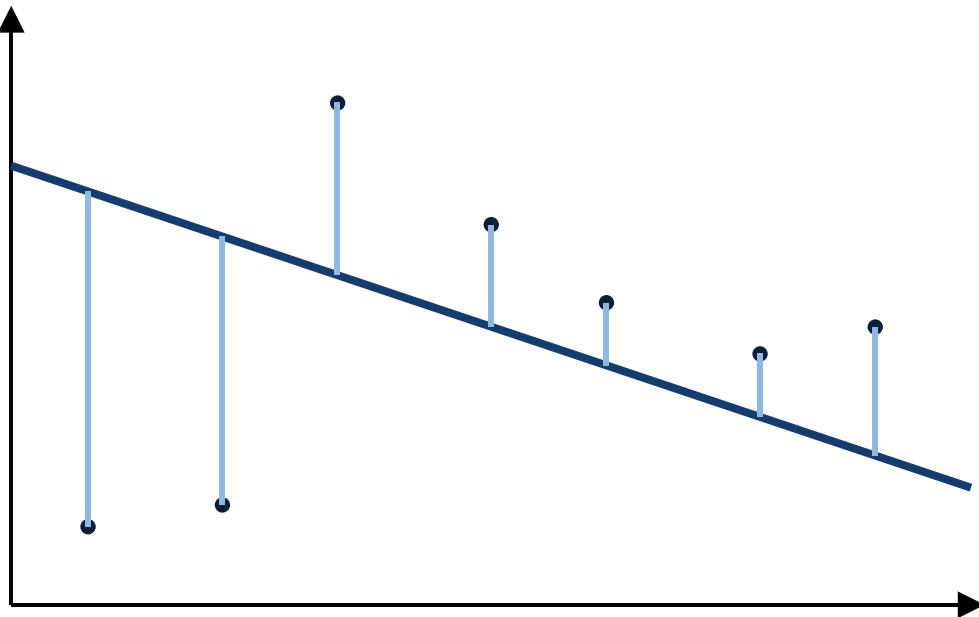
What Makes a Line the “Best” Fit?

Many lines can pass near the same data — but only one minimizes the total distance between the line and every point. Compare three candidate fits below:

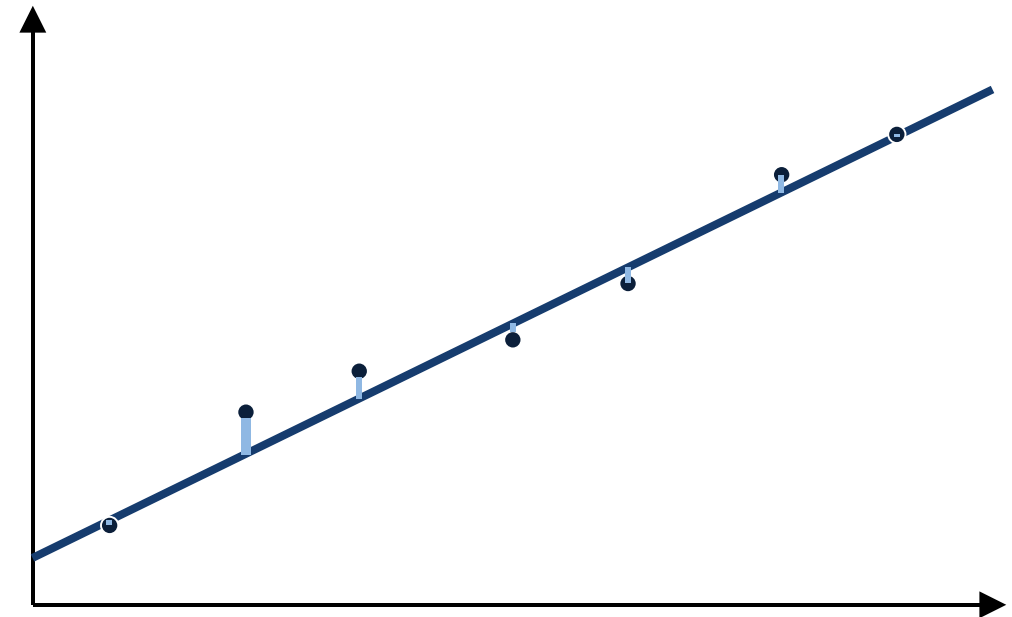


The Error Function

The error function is a metric that tells us how well a model is doing, based on the difference between the **actual value** and the **predicted value** for each point.



Large error



Small error

Absolute Error

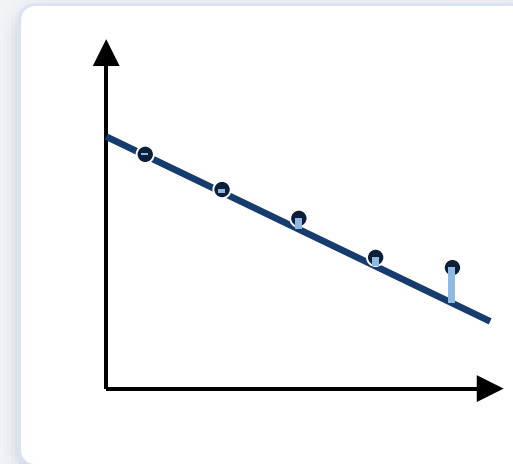
The error is the difference between the predicted value and the actual value for each data point:

$$\text{Error} = |y'(i) - y(i)|$$

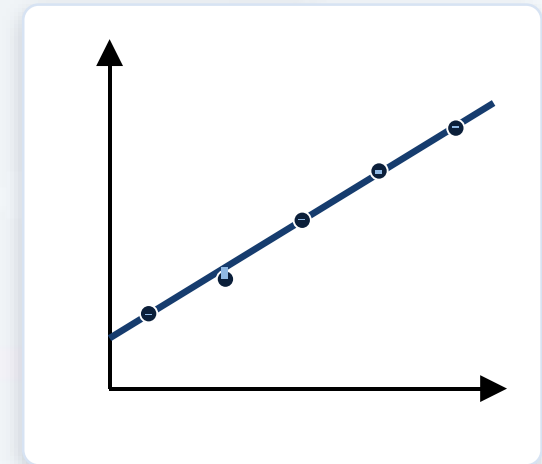
$y'(i)$ the predicted value

$y(i)$ the actual (true) value

Taking the absolute value always turns the error into a positive number.



Large absolute error



Small absolute error

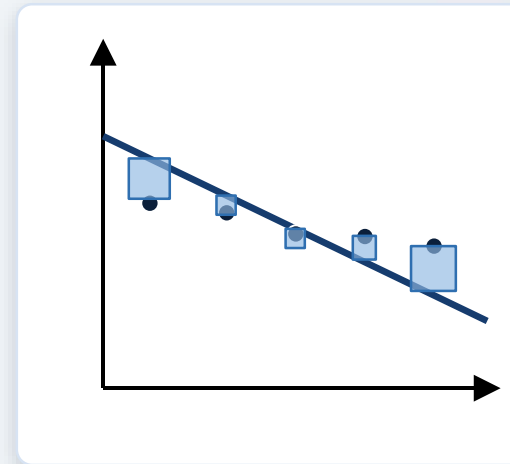
Absolute error treats every point equally, regardless of how large the miss is.

Squared Error

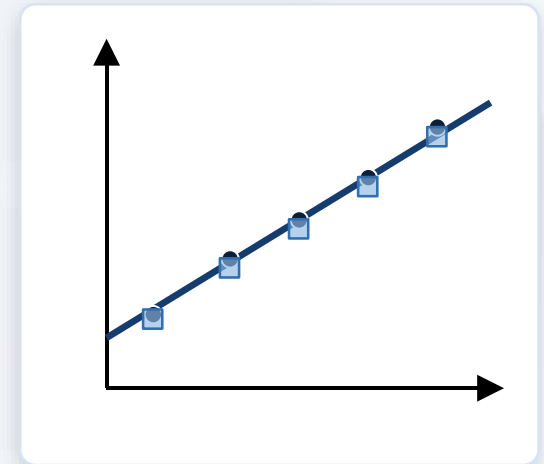
Similar to absolute error, but instead of taking the absolute value of the difference, we take the **square**. This also always produces a positive number — and penalizes larger mistakes more heavily.

$$\text{Error} = (y'(i) - y(i))^2$$

Squared error is used far more commonly in practice than absolute error.



Large squared error



Small squared error

Notice how the squares grow quickly for bigger misses — squaring punishes outliers.

Cost Function = Total Error

To get the overall error for the model, we average all of the squared errors across every data point. This average is the **Mean Squared Error (MSE)** — our cost function.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Average squared distance between every predicted point and its true value

Training a linear regression model means finding the weight & bias that make MSE as small as possible.

Mean Absolute Error (MAE)

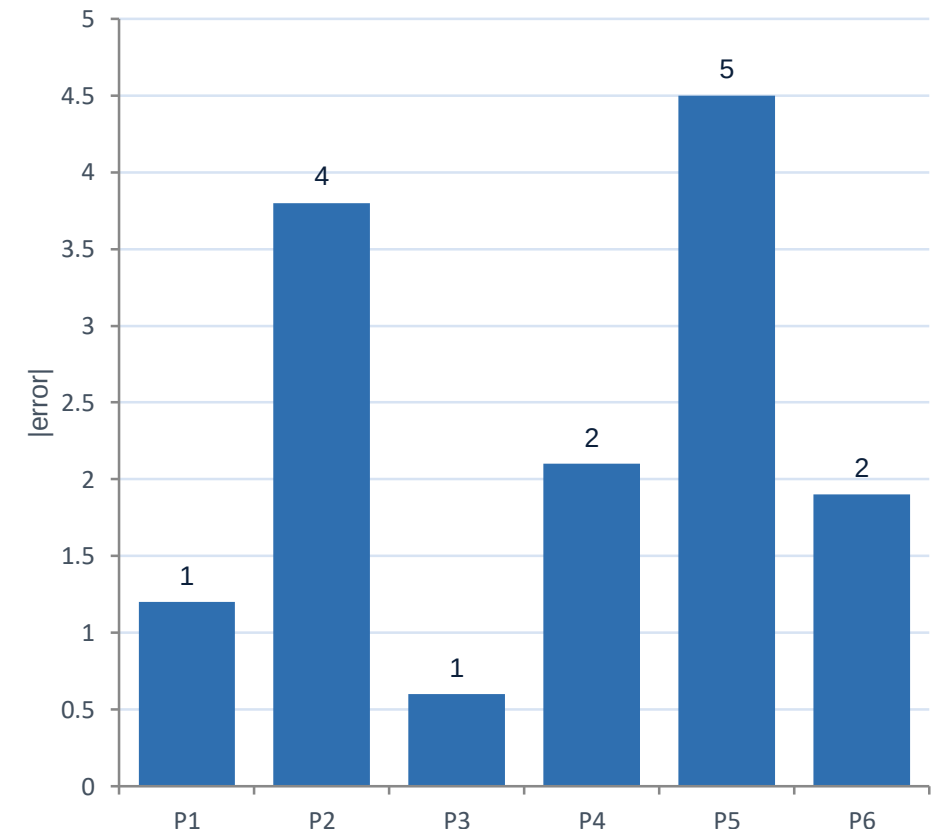
AVERAGING THE ABSOLUTE ERROR

$$\text{MAE} = 1/n \cdot \sum |y'(i) - y(i)|$$

- MAE is the average of all absolute errors across the n data points.
- Same unit as the target variable, so it reads directly (e.g. “off by \$3,200”).
- Every point is weighted equally — a 10-unit miss counts exactly 10x a 1-unit miss.
- Because it isn't smooth at zero, gradient-based training favors MSE instead.

Rule: pick MAE when outliers are noise you don't want to overweight — it gives a robust, easy-to-explain typical error.

Per-point absolute error — every miss counts equally



Notice the tallest bar (P5) contributes proportionally — not exponentially — to the average.

Root Mean Squared Error (RMSE)

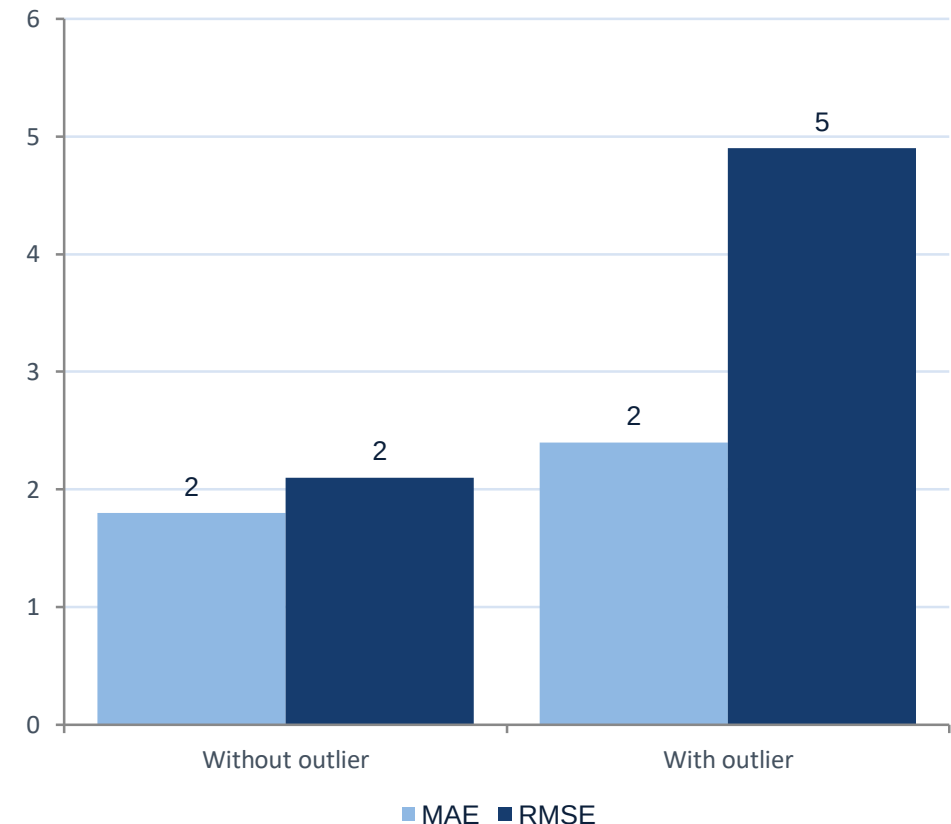
BRINGING MSE BACK TO REAL UNITS

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\left(\frac{1}{n} \cdot \sum (y'(i) - y(i))^2 \right)}$$

- RMSE is simply the square root of the cost function, MSE.
- Squaring inside still penalizes big misses heavily — one bad prediction can dominate.
- The square root undoes the squared units, so RMSE is comparable to the target again.
- RMSE is always $\geq$ MAE for the same dataset; the gap grows as errors get less uniform.

Rule: pick RMSE/MSE when large errors are genuinely more costly — e.g. safety-critical or financial forecasts.

One outlier moves RMSE far more than MAE



A single large miss barely moves MAE, but it drags RMSE upward — that sensitivity is the whole point of squaring.

R^2 — Coefficient of Determination

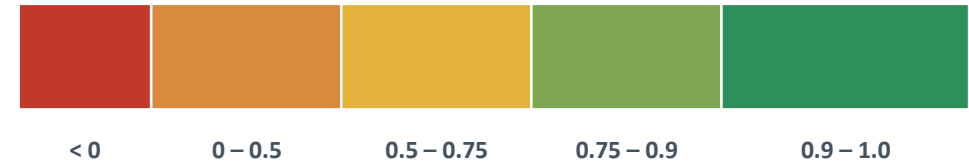
HOW MUCH VARIANCE THE MODEL EXPLAINS

$$R^2 = 1 - SS_{\text{res}} / SS_{\text{tot}}$$

- SS_{res} is the model's squared error; SS_{tot} is the error of always predicting the mean.
- $R^2 = 1$ means perfect predictions; $R^2 = 0$ means “no better than guessing the mean.”
- Unlike MAE/RMSE, R^2 is unit-free — a single number to compare across problems.
- R^2 can go negative when the model fits worse than a flat average line.

Rule: use R^2 to communicate overall fit quality; pair it with RMSE/MAE to see the actual error size.

Reading the R^2 scale



- Worse than the mean
- Weak fit
- Moderate fit
- Good fit
- Excellent fit

Example: $R^2 = 0.85$ means the model explains 85% of the variance in the target — the remaining 15% is unexplained error.

Choosing the Right Regression Metric

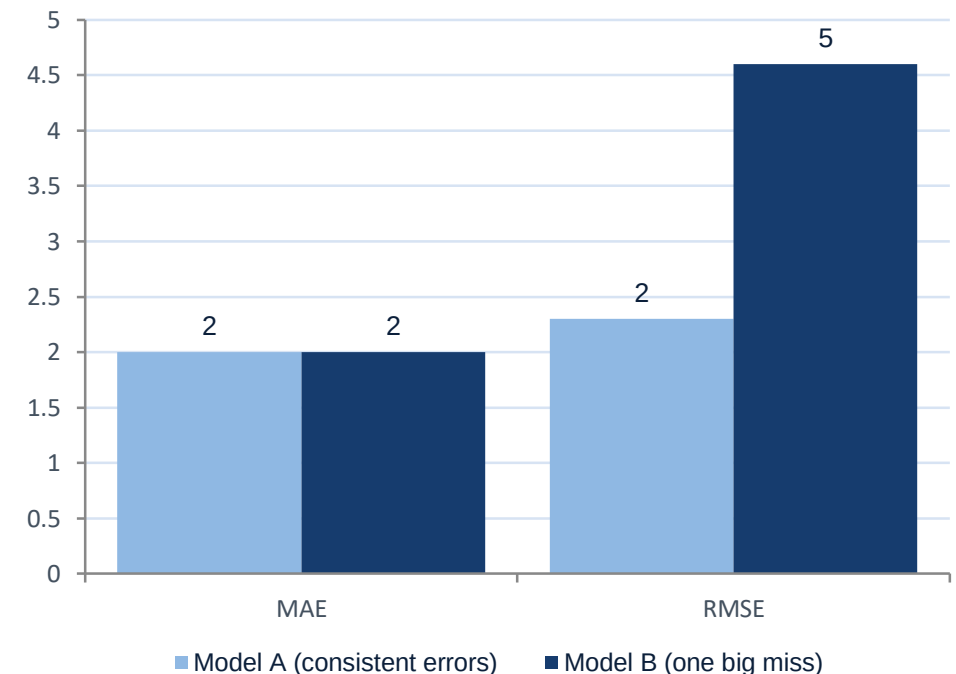
MAE • MSE • RMSE • R² SIDE BY SIDE

Metric	Formula	Units	Outlier sensitivity
MAE	$1/n \cdot \sum y' - y $	Same as target	Low
MSE	$1/n \cdot \sum (y' - y)^2$	Squared target	High
RMSE	$\sqrt{\text{MSE}}$	Same as target	High
R ²	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Unit-free (0–1)	Moderate

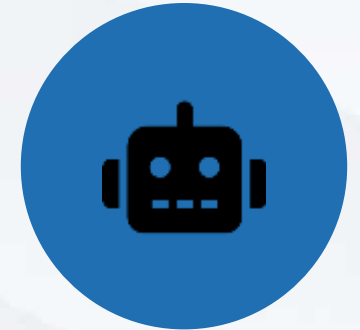
Decision rules

- Need a result stakeholders can sanity-check in real units? Report MAE or RMSE.
- Training a model with gradient descent? Optimize MSE — it's smooth and differentiable.
- Outliers are data errors, not signal? Favor MAE so they don't dominate the score.
- Large misses are genuinely more costly? Favor RMSE/MSE to penalize them harder.
- Comparing fit quality across different datasets or targets? Report R².

Same errors, four different lenses



Both models share the same MAE, but Model B's single large miss is exposed by RMSE — the metric you choose changes the story.



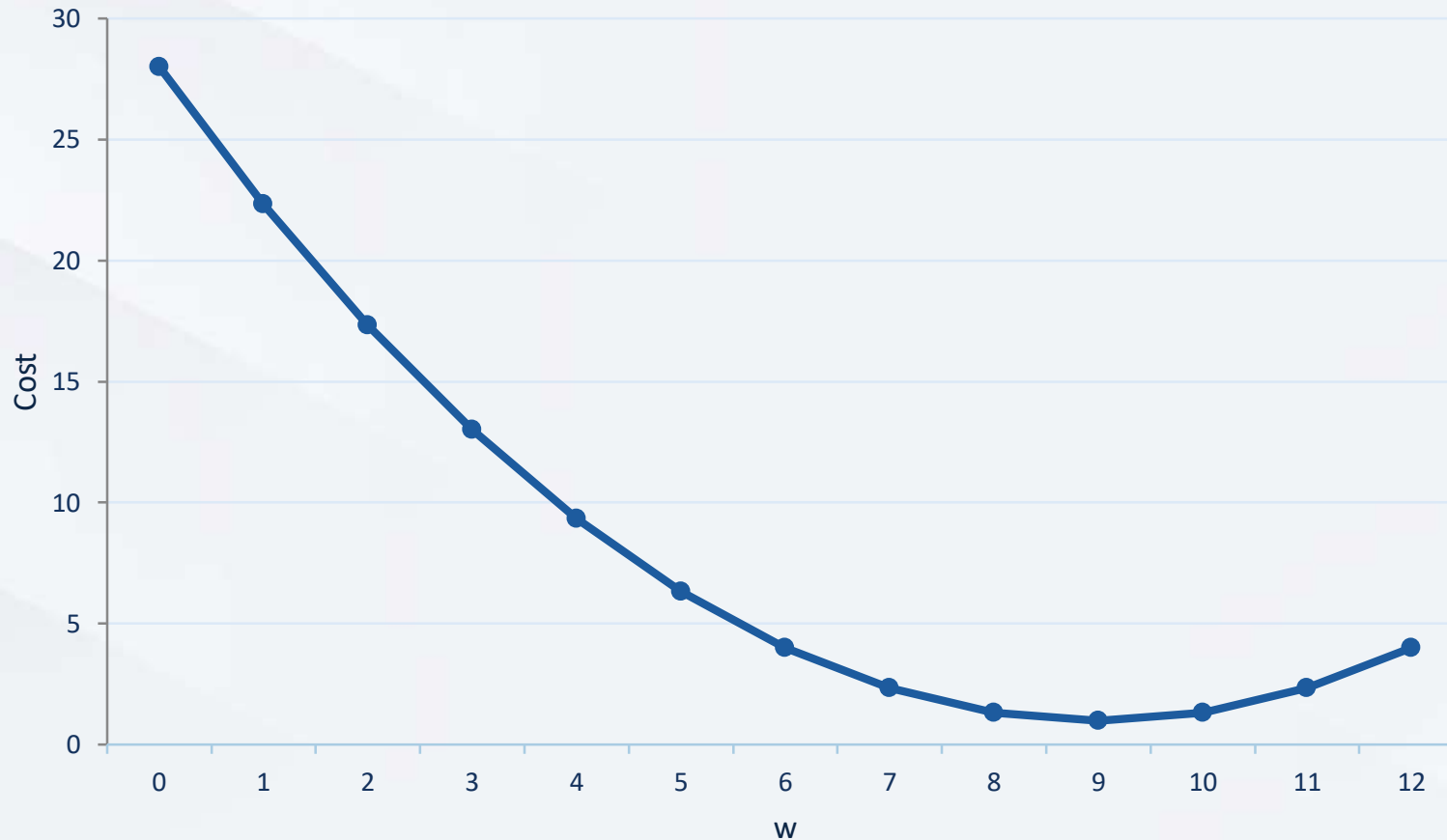
Optimizing the Model

Gradient Descent

An **optimization algorithm** used to minimize the cost function by adjusting the model's parameters in the direction that reduces the cost.

Keep changing $\mathbf{w}$, $\mathbf{b}$ to reduce the cost function $J(\mathbf{w}, \mathbf{b})$ until we settle at or near a minimum.

Descending the Cost Curve



RANDOM INITIAL VALUE

Gradient descent starts from an arbitrary starting point on the curve.

MINIMUM

Each learning step moves w closer to the point where cost is lowest.

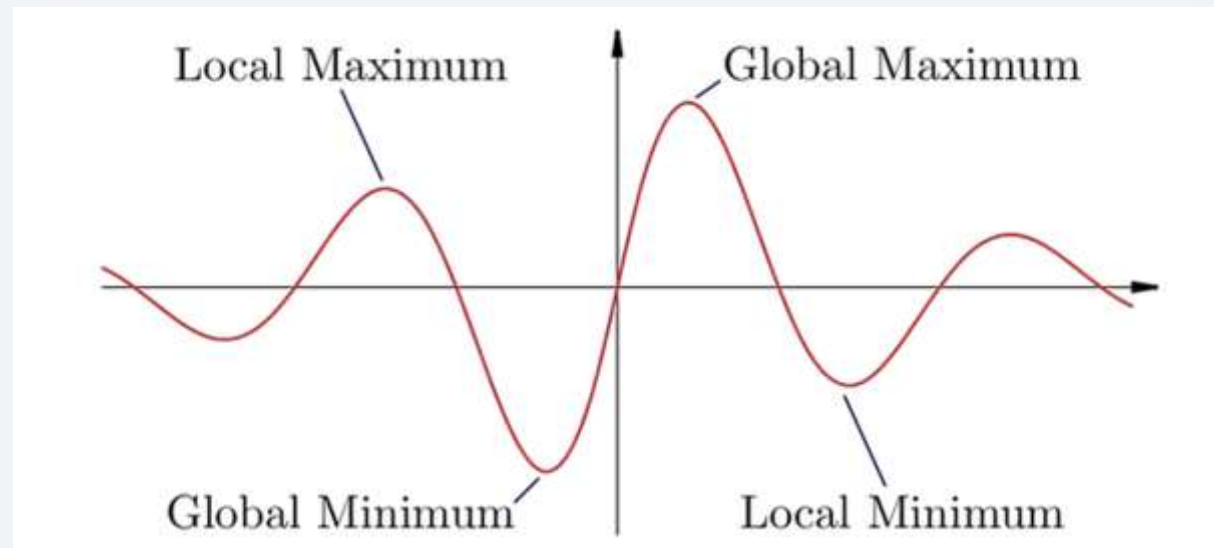
Global Minima vs. Local Minima

GLOBAL MINIMA

The lowest point of the entire cost function — the best possible set of parameters.

LOCAL MINIMA

A point lower than its neighbors, but not necessarily the lowest in the entire function.



The Gradient Descent Algorithm

```
repeat until convergence {
```

$$\mathbf{w} = \mathbf{w} - \alpha \partial J(\mathbf{w}, \mathbf{b}) / \partial \mathbf{w}$$

$$\mathbf{b} = \mathbf{b} - \alpha \partial J(\mathbf{w}, \mathbf{b}) / \partial \mathbf{b}$$

```
}
```

α = learning rate $\partial J / \partial \mathbf{w}, \partial J / \partial \mathbf{b}$ = gradients of the cost function

What Is Convergence?

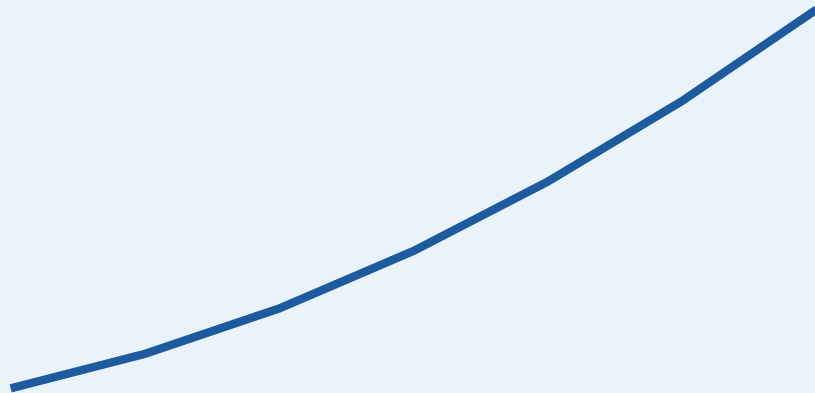
We reach convergence when $\mathbf{w}$ and $\mathbf{b}$ are not changing significantly anymore.

Why that means we've found a minimum

When w and b stop changing, we've reached a point lower than the points near it — a global or local minimum.

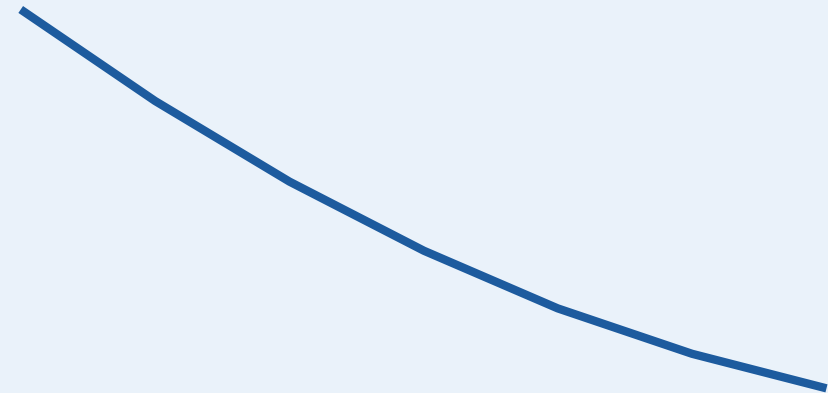
Why We Compute the Derivative

Positive Slope ($\partial J / \partial w > 0$)



The new value of w is smaller — we step to the left
"smaller w ".

Negative Slope ($\partial J / \partial w < 0$)



The new value of w is bigger — we step to the right
"bigger w ".

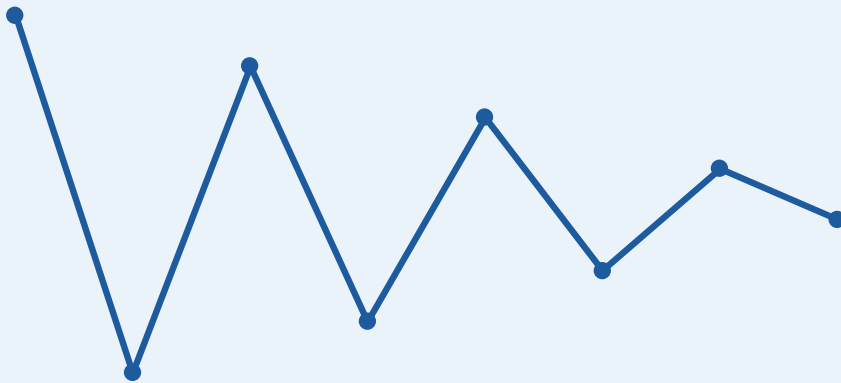
Learning Rate

One of the most important **hyperparameters** in optimization algorithms.

It controls how big a step the algorithm takes when updating the model's parameters — the speed at which the algorithm learns.

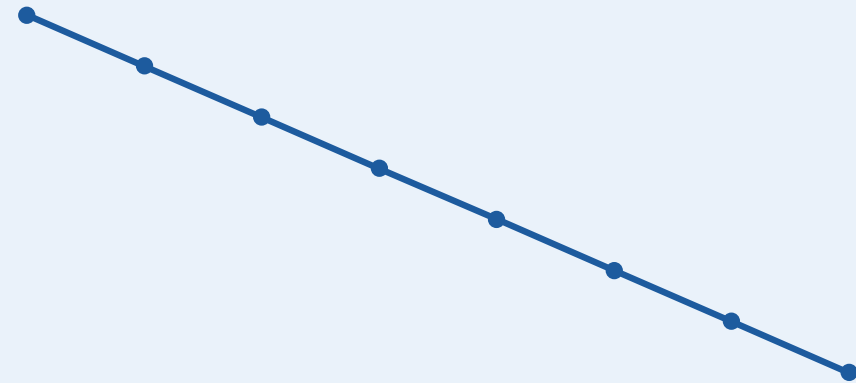
Big vs. Small Learning Rate

BIG LEARNING RATE



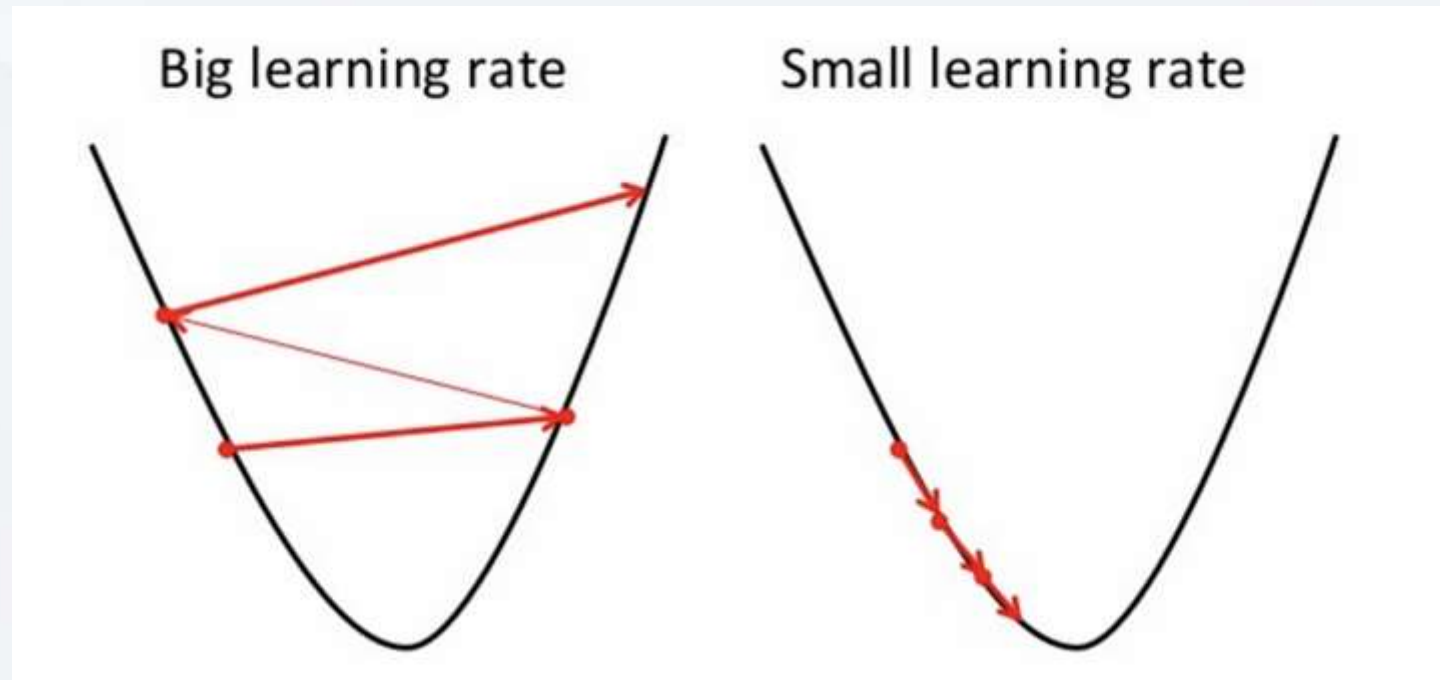
Overshoots the minimum — zig-zags and may never settle.

SMALL LEARNING RATE



Steady, controlled steps toward the minimum.

Big vs. Small Learning Rate



Choosing the Learning Rate

TOO LARGE

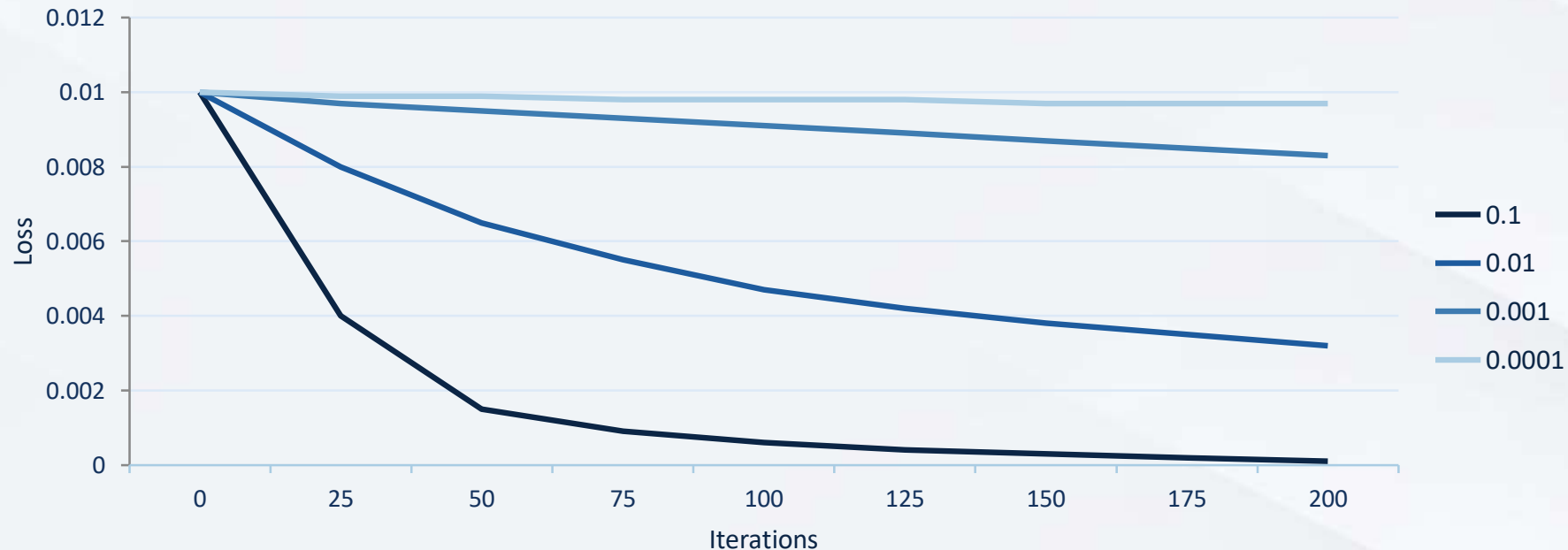
Updates might be too big, causing the algorithm to overshoot the optimal solution. This can lead to divergence.

TOO SMALL

Updates are very tiny, making the algorithm slow to converge — or causing it to get stuck in a local minimum.

Finding the Optimal Learning Rate

- 1 Try a range of learning rates and track the loss function
- 2 Plot loss vs. learning rate to see where loss decreases fastest before it spikes

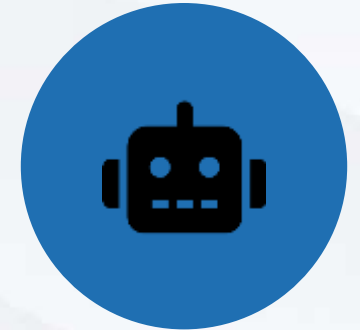


So, In Gradient Descent...



Linear Regression

Core Assumptions



Linear Regression Assumptions

1. Linearity

2. Homoscedasticity

3. Independence of Errors

4. Normality of Residuals

5. No Multicollinearity

Linear Regression (Core Assumptions)

1. Linearity

- **What it means:** The relationship between the independent variables (X) and the dependent variable (Y) must be linear.

2. Homoscedasticity

3. Independence of Errors

4. Normality of Residuals

5. No Multicollinearity

- **Why it matters:** If you fit a straight line to curvilinear data, your model will underfit and produce highly inaccurate predictions.
- **How to test:** Examine a scatter plot of X vs. Y, or a residual plot (residuals vs. predicted values). Look for random scatter; a U-shape or wave pattern indicates non-linearity.

Linear Regression (Core Assumptions)

1. Linearity

2. Homoscedasticity

3. Independence of Errors

4. Normality of Residuals

5. No Multicollinearity

(Constant Residuals Variance)

- **What it means:** The variance of the residuals must remain constant across all levels of the independent variables.
- **Why it matters:** Heteroscedasticity (varying variance) inflates or deflates standard errors, making your hypothesis tests and p-values deceptive.
- **How to test:** Check a plot of residuals against predicted values. If the spread of data points fans out like a cone or funnel, homoscedasticity is violated. You can also run a formal statistical test like the Breusch-Pagan test.

Linear Regression (Core Assumptions)

1. Linearity

2. Homoscedasticity

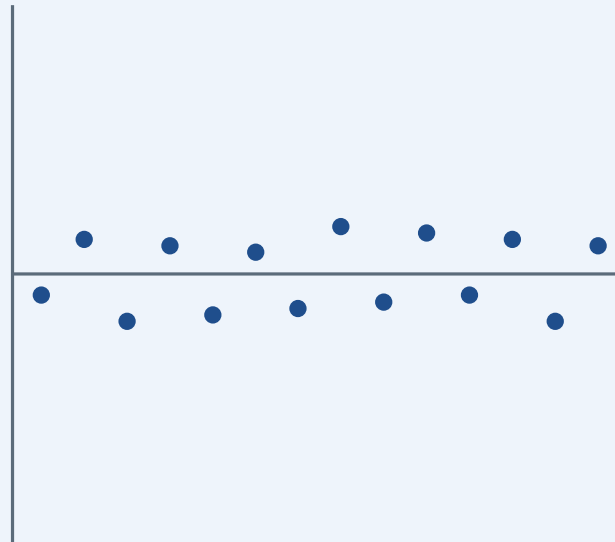
3. Independence of
Errors

4. Normality of
Residuals

5. No Multicollinearity

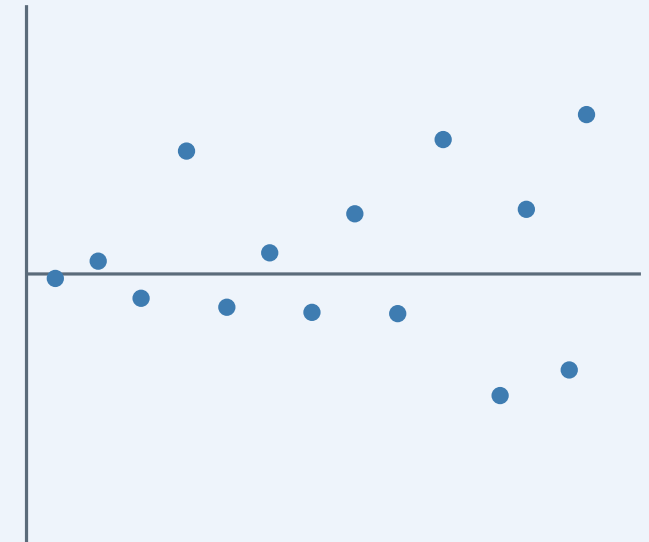
(Constant Residuals Variance)

Homoscedastic



Constant spread → OK

Heteroscedastic



Fan-out pattern → violated

Linear Regression (Core Assumptions)

1. Linearity

2. Homoscedasticity

3. Independence of Errors

4. Normality of Residuals

5. No Multicollinearity

No Autocorrelation

- **What it means:** The residuals of one observation must not be correlated with the residuals of another observation.
- **Why it matters:** This issue is highly common in time-series data, where yesterday's values heavily influence today's. Autocorrelation artificially inflates the model's statistical significance.
- **How to test:** Use the Durbin-Watson test. A test score near 2.0 indicates no autocorrelation, while scores far below or above 2.0 flag a violation.

Linear Regression (Core Assumptions)

1. Linearity

2. Homoscedasticity

3. Independence of Errors

4. Normality of Residuals

5. No Multicollinearity

- **What it means:** The error terms (residuals) should be normally distributed around a mean of zero. The input data (X and Y) does not need to be normally distributed — only the errors do.
- **Why it matters:** While the central limit theorem minimizes this issue for massive sample sizes, normality is strictly required to calculate accurate confidence intervals and p-values in smaller datasets.
- **How to test:** Inspect a Quantile-Quantile (Q-Q) plot. If the residuals align tightly along the diagonal reference line, they are normally distributed. Alternatively, run a Shapiro-Wilk test.

Linear Regression (Core Assumptions)

1. Linearity

- **What it means:** In multiple linear regression, the independent variables should not be highly correlated with one another.

2. Homoscedasticity

3. Independence of Errors

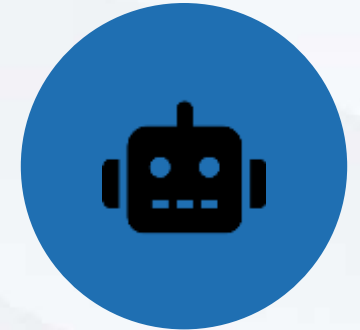
4. Normality of Residuals

5. No Multicollinearity

- **Why it matters:** Severe multicollinearity makes it impossible for the model to isolate the true, individual impact of each independent variable on the target variable.
- **How to test:** Calculate the Variance Inflation Factor (VIF) for each predictor. A VIF value greater than 5 or 10 indicates problematic multicollinearity that requires fixing.

Regularization

A technique used in machine learning to reduce overfitting and help models generalize to unseen data.



1 What Is Overfitting?

A model overfits when it learns the training data too well — including its noise and unnecessary details.

Training Performance

▲ **Very Good**

The model fits the exact examples it was trained on.

Test / Validation Performance

▼ **Poor**

The model fails to generalize to data it has not seen.

GOAL OF REGULARIZATION

Make the model simpler so it can generalize better to unseen data.

2 The Main Idea

Regularization adds a penalty for model complexity to the loss function.

Normally

$$Loss = Error$$

With Regularization

$$Loss = Error + Regularization Penalty$$

So the model tries to balance two goals:

1 Make predictions accurate.

2 Keep the model parameters relatively small and simple.

3

L1 Regularization

L1 adds the absolute values of the weights:

$$Loss = Error + \lambda \sum |w_i|$$

w_i = model weights λ = regularization strength

Main effect: L1 can make some weights exactly zero — this performs feature selection.

EXAMPLE

Before: $w = [2.5, 0.8, 0.01, 3.2]$

After L1: $w = [2.3, 0, 0, 3.0]$

REMEMBER

L1

Sparse weights
Feature selection

4

L2 Regularization

L2 adds the squared values of the weights:

$$Loss = Error + \lambda \sum w_i^2$$

Main effect: L2 makes weights smaller, but usually not exactly zero.

L2 is also called “weight decay” in many deep-learning contexts.

EXAMPLE

Before: $w = [5, 2, 0.5]$

After L2: $w \approx [3.8, 1.6, 0.4]$

Values shrink toward zero, but none of them become exactly zero.

REMEMBER

L2

Smaller weights

Smoother, simpler model

5

L1 vs L2 Regularization

	L1 (Lasso)	L2 (Ridge)
Penalty	$\sum w_i $	$\sum w_i^2$
Effect on weights	Can become exactly 0	Become small
Feature selection	Yes	Usually no
Produces sparse model	Yes	Usually no
Main idea	Remove unnecessary features	Reduce large weights

Easy memory trick: L1 → “Leave some weights at 0” L2 → “Lower the weights”

6 The Role of λ (Lambda)

λ controls how strong the regularization is.

Small λ

Weak regularization

$$\text{Loss} = \text{Error} + \text{small penalty}$$

The model can become more complex $\rightarrow$
possible overfitting.

Large λ

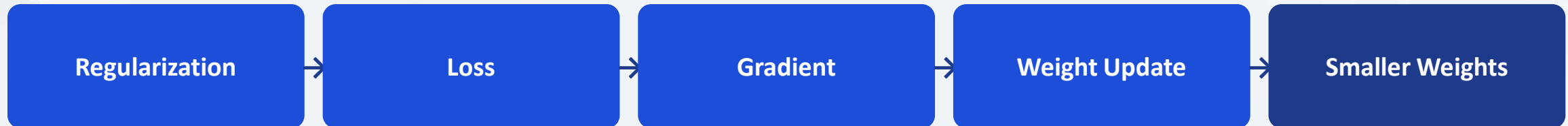
Strong regularization

$$\text{Loss} = \text{Error} + \text{large penalty}$$

The model becomes very simple $\rightarrow$ possible
underfitting.

7 How Regularization Changes the Weights

The penalty term changes the loss value directly. Since the optimizer minimizes the loss, it changes the weights indirectly.



7 How Regularization Changes the Weights

For L1, the gradient gains an extra term:

$$\partial Loss / \partial w_i = \partial Error / \partial w_i + \lambda \cdot \text{sign}(w_i)$$

Then the optimizer updates the weight:

$$w_i \leftarrow w_i - \eta \cdot \partial Loss / \partial w_i$$

SIMPLE EXAMPLE

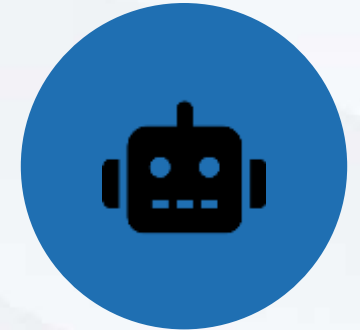
$$w = 5, \lambda = 0.1 \rightarrow \lambda |w| = 0.5$$

The loss gets an extra +0.5.

$$\partial(\lambda |w|) / \partial w = \lambda$$

Regularization doesn't directly set the weights — it changes the loss landscape and gradient, pushing the optimizer toward smaller weights. For L1 this can push a weight exactly to 0; for L2 it only pushes weights toward 0.

Polynomial Regression

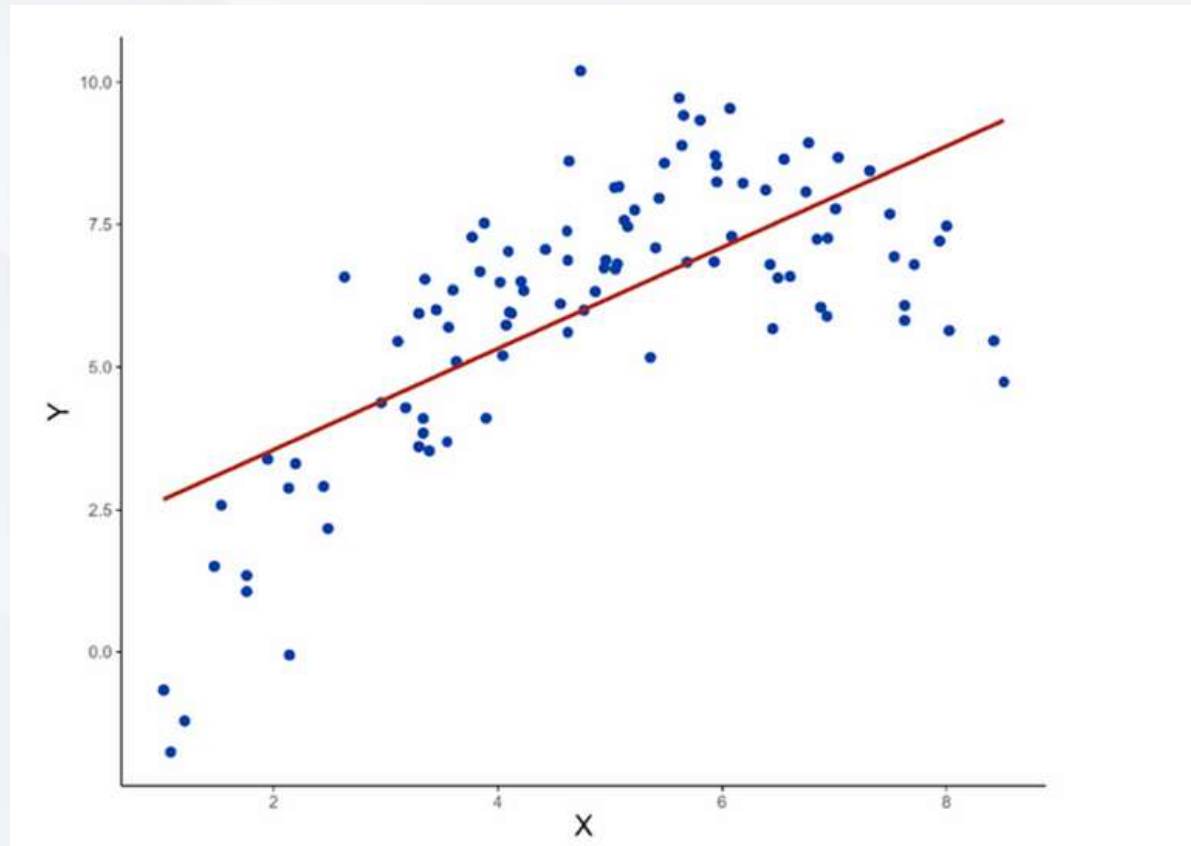


Polynomial Regression

A type of linear regression where the relationship between the input variable and the output is modeled as an **n-th degree polynomial**.

It allows us to capture curved (non-linear) patterns in the data using a linear model on transformed features.

When a Straight Line Isn't Enough

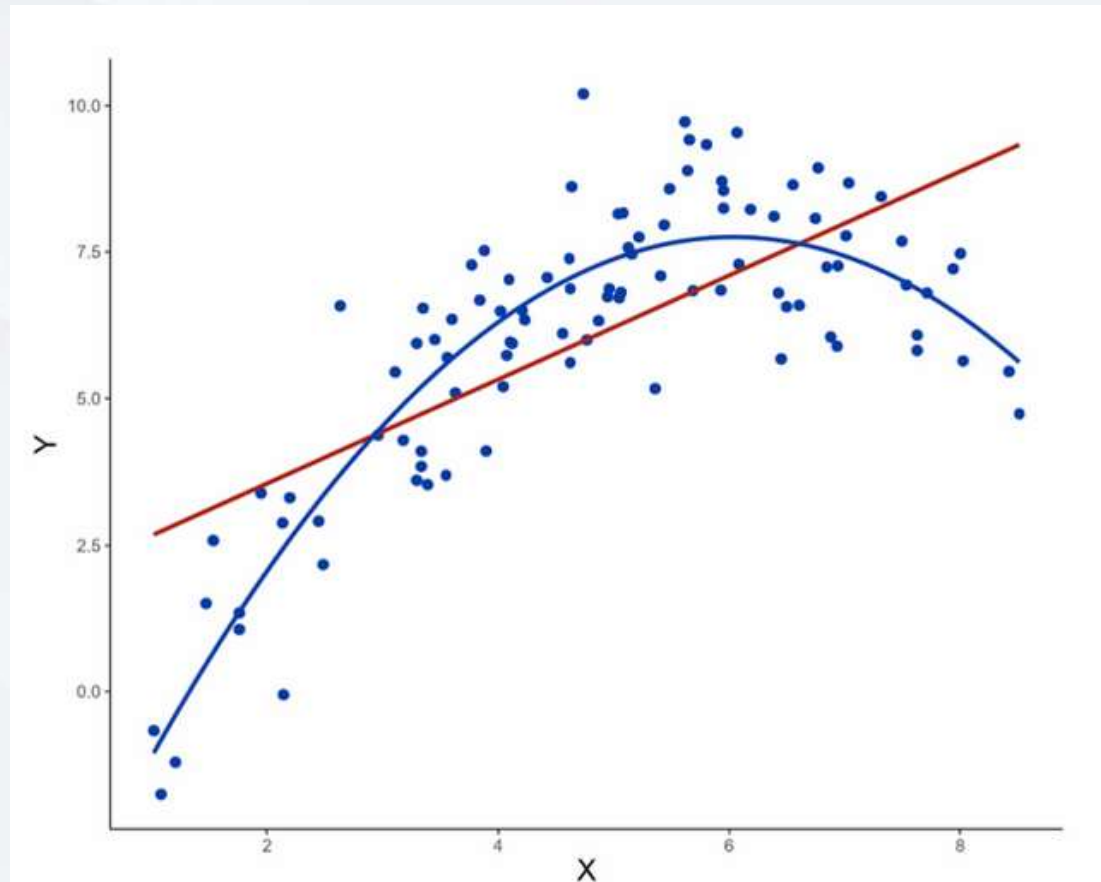


OBSERVATION

The linear line does not fit the data properly — it misses the curve in the pattern.

$$Y \sim a + bX + \varepsilon$$

A 2nd-Degree Polynomial Fits Better



BETTER FIT

Increasing the equation to a 2nd-degree polynomial models the curve better and gives a more accurate fit.

$$Y \sim a + bX + bX^2 + \epsilon$$

Thank you